

PRODUCT DOCUMENTATION

Word Cloud Studio

User guide & technical documentation — turn text, documents and datasets into beautiful, customisable word clouds with real frequency analysis, seven shapes plus your own silhouettes, and high-resolution exports up to 4K.

Creator	**Dr. Asif Ahmad**
Product	Word Cloud Studio — version 1.0
Category	Design
Published by	Asif SaaS Studio
Date	20 October 2026
Format	Single-file web application (index.html, 47.9 KB)

Private by design: every file is parsed and rendered on the user's own device — no uploads, no accounts, no tracking. And honest by design: version 1.0 is frequency analysis, not AI, and this guide says exactly where the line sits (Chapter 11).

Contents

1. What this product does — and who it serves
2. The four-step workflow
3. Getting text in — paste and file formats
4. Frequency analysis — tokenising, stop words, phrases
5. The layout engine — spiral placement without collisions
6. Shapes — seven masks and your own silhouette
7. Design controls — fonts, colour, background, spacing
8. Editing words and reading the frequency table
9. Exports — PNG, JPG, SVG, PDF and CSV, up to 4K
10. Research honesty — frequency is not a theme
11. The AI roadmap and deployment plan
12. Publishing and SEO notes
13. How correctness was verified
14. Product information and credits
15. Appendix A — the analysis & layout core, verbatim
16. Appendix B — the file-parsing and export cores, verbatim

1. What this product does — and who it serves

Word Cloud Studio turns any text — a speech, an article, survey answers, a document, a spreadsheet of keywords — into a word cloud whose word sizes are driven by counted frequency, laid out without overlaps inside a chosen shape, styled with fonts and colour themes, and exported at up to 4K resolution for slides, papers, posters and social media. Everything runs in the browser: the files a user opens never leave their device.

Audience	Typical use
Teachers & students	Vocabulary visualisations, lesson openers, essay and thesis figures.
Researchers	A first visual pass over open-ended survey responses, interview transcripts and abstracts — a starting point for analysis, never a substitute for it (Chapter 10).
Presenters & marketers	Title slides, event backdrops, campaign and brand clouds in house colours.
Bloggers & designers	Article art, book covers, social posts — including clouds shaped by a logo or product silhouette.

The product's advantage is the combination in one free tool: genuine frequency analysis (with phrase detection), a collision-free layout engine, multilingual tokenising (English, Urdu, Arabic and any Unicode script), design control, and high-resolution export — with no sign-up and no server.

2. The four-step workflow

Step	What happens
1 · Enter text or upload	Type or paste into the text box, press Sample text to explore, or upload a TXT, MD, CSV, DOCX, PDF or XLSX file. The file is parsed on the device and its text loads into the box.
2 · Analyse words	The text is cleaned and tokenised; frequencies are counted; optional filters remove stop words, numbers and very short words; two-word phrases are detected. Statistics (total words, unique words, words placed) update live.
3 · Customise design	Shape, maximum words, font, weight, orientation, colour theme or custom colours, background, and word spacing — every change re-lays the cloud immediately. Individual words can be highlighted, enlarged, removed or added in the frequency table.
4 · Preview and export	Click any word in the preview to see its exact count and share. Download PNG, JPG, SVG or PDF at 960, 1920, 2560 or 3840 px, or copy/download the frequency data as CSV.

3. Getting text in — paste and file formats

All parsing happens inside the page, in the IO core reproduced in Appendix B. No file is uploaded to any server at any point.

Format	How it is read
Paste / type	Directly into the text box; the cloud updates as you type.
.txt / .md / .csv	Read as plain text. For CSV keyword datasets, the words and numbers are tokenised like any other text — a column of terms with counts therefore weights the cloud naturally.
.docx	The page opens the file as a ZIP (stored and deflated entries, inflated with the browser's DecompressionStream) and extracts the text of `word/document.xml`.
.xlsx	The same ZIP reader opens the workbook, resolves shared strings, reads the first worksheet and joins its cell values into text.
.pdf	Content streams are inflated (Flate, and ASCII85+Flate) and the text-showing operators are read. See the honesty note below.

> **The honest PDF caveat.** A PDF whose text is drawn with subset embedded fonts — or a scanned PDF that is really a photograph of a page — contains no readable text layer for any in-page reader. Word Cloud Studio detects this and tells the user plainly, asking them to copy the text from their PDF reader and paste it, instead of rendering a cloud of garbage characters. This is the same honesty standard as the rest of the studio: a clear message beats a silent failure.

Legacy binary formats (old .doc and .xls) are likewise not guessed at: the app reports that the file type cannot be read and suggests saving as .docx/.xlsx or .txt first.

4. Frequency analysis — tokenising, stop words, phrases

4.1 Tokenising, including Urdu and Arabic

A word is a run of Unicode letters and marks, with an optional inner apostrophe (`don't` stays one word); numbers form separate tokens. Because the tokeniser is defined over Unicode letter classes rather than the Latin alphabet, Arabic-script text tokenises correctly: in verification, the Urdu word کہانا and the Arabic word العلم were each counted three times from real test strings. Words are lower-cased for counting where the script has case. Right-to-left words are drawn with the browser's text engine, and a Noto Nastaliq Urdu / Arabic font option is offered (used when installed on the device).

4.2 Filters

Control	Effect
Remove common stop words	Drops the built-in stop lists: ~180 English words, plus Urdu and Arabic lists, merged into one lookup set. On by default.
Remove numbers	Drops pure-number tokens (years, counts) so they do not dominate the cloud. On by default.
Minimum word length	Slider-filtered: words shorter than the chosen length (after apostrophes are stripped) are excluded.
Maximum words	Caps how many of the top entries enter the layout (the rest remain in the frequency table and CSV).

4.3 Two-word phrase detection

With phrase detection on, every adjacent pair of kept words is counted as a bigram. A pair that occurs at least twice is promoted to an entry of its own — so a text about food safety yields the single entry **"food safety"** alongside (and ranked against) the single words, instead of splitting the concept into "food" and "safety". Phrases are flagged in the frequency table. In verification, "food safety" was counted three times in a test paragraph, and the phrase "science learning" twice in the prototype text.

5. The layout engine — spiral placement without collisions

The engine (Appendix A) places words largest-first, because big words are the hardest to fit. Font size scales with frequency on a softened curve (exponent 0.55) between a minimum and maximum derived from the canvas, so a word used four times as often is prominent without crushing the rest. Each word's box is measured with the real canvas text measurer at its actual font, then the engine searches outward from the centre along a spiral — slightly elliptical, matching the canvas — testing each candidate position for two things:

- **Shape test** — the whole box must lie inside the selected mask (Chapter 6);
- **Collision test** — the box must not intersect any already-placed box, with the user's spacing (padding) added on all sides.

The first position that passes both tests wins. Orientation is applied per word before measuring: all-horizontal, all-vertical, or a seeded mix (about 28% vertical). The layout uses a fixed-seed generator, so the same text and settings reproduce the same cloud; the Regenerate button advances the seed for a fresh arrangement.

> **"Placed X of Y words" — said out loud.** In a tight shape (a star's points, a small heart) some low-frequency words physically cannot fit at a legible size. Rather than silently dropping them or shrinking them to dust, the app reports exactly how many of the selected words were placed — and the unplaced words remain visible in the frequency table and the CSV export. Raising the export size, reducing the maximum-words count, or widening the spacing trade-offs are the user's levers, and the app does not hide that they exist.

6. Shapes — seven masks and your own silhouette

Shape	Definition
Rectangle	The full canvas — free-form clouds.
Circle	Analytic ellipse equation.
Heart	The classic implicit heart curve, fitted to the canvas.
Star	A ten-vertex star polygon (point-in-polygon test).
Cloud	A union of overlapping circles forming a cloud outline.
Lightbulb	A bulb circle over a base rectangle.
Tree	A triangular canopy over a trunk rectangle.
Custom silhouette	Your own image — see below.

For a custom silhouette, the uploaded image (a logo, a product outline, a map) is drawn onto a small 192 × 120 analysis grid. Each cell is marked "inside" when the image there is sufficiently dark or opaque — precisely, when `alpha × (1 − luminance) > 0.45` — so dark artwork on a transparent or light background becomes the mask. Word boxes must then fit entirely within inside cells. The silhouette image itself is processed locally and never uploaded.

7. Design controls — fonts, colour, background, spacing

Control	Options
Font	Georgia, Arial, Verdana, Impact, Times New Roman, Courier New, and a Noto Nastaliq Urdu / Arabic option for Arabic-script clouds.
Weight	Bold or normal.
Orientation	Horizontal · Mixed (some vertical) · Vertical.
Colour themes	Seven preset palettes of five colours each — Ocean Blue, Studio Emerald, Sunset, Berry, Royal Gold, Mono Ink, Festival — assigned across words in rank order with a seeded variation; or **Custom colours**: three pickers of your own (brand colours, university colours), cycled across the cloud.
Background	White · Studio paper · Emerald · Charcoal · Transparent. Transparent applies to PNG and SVG export (a JPG or PDF page is necessarily opaque — white is used, and the interface says so).
Spacing	A padding slider controlling the minimum gap between word boxes — tighter packs more words, looser breathes.

Every control re-lays the cloud live; there is no "apply" step. Highlighted words (Chapter 8) are always drawn in studio gold so they survive any theme.

8. Editing words and reading the frequency table

Below the preview, the full frequency table lists every entry — word or phrase, count, and share — with three actions per row:

Action	Effect
★ Highlight	The word is drawn in gold (#C9A227) regardless of theme — for the one term the audience must notice.
+ Boost	Multiplies the word's display size (×1.25 per boost) without changing its true count — the table still shows the honest number.
× Remove	Excludes the word from the cloud (it stays out until the text is re-analysed).
Add a word	A form adds any word with a chosen count — a sponsor name, a hashtag, a term the text implies but never states.

Clicking any word in the cloud itself shows its exact readout beneath the preview — "food safety" — 12 occurrences (8.3% of kept words) — so a presenter can answer "how big is that, really?" without leaving the page.

9. Exports — PNG, JPG, SVG, PDF and CSV, up to 4K

Format	How it is produced
PNG	The cloud is re-rendered on an off-screen canvas at the chosen width and encoded by the page's own zlib/PNG writer. Sizes: 960 (web), 1920 (Full HD 1080p), 2560 (2K), 3840 px (4K / print).
JPG	The same high-resolution render, composited on the background colour (JPEG has no transparency).
SVG	A true vector file: every word is a real `<text>` element with its font, size, rotation and colour — infinitely scalable, and the words remain selectable, searchable text in the file.
PDF	A one-page PDF written by the in-page image writer: the high-resolution render embedded as an image XObject with a correct cross-reference table — opens in any PDF reader, sized to the image's aspect.
CSV	The complete frequency data (word, count) — copied to the clipboard or downloaded, for spreadsheets and further analysis.

Transparent exports: with the Transparent background selected, PNG keeps a genuine alpha channel and SVG omits the background rectangle entirely — the cloud drops cleanly onto slides and web pages. Layout is resolution-independent: the same placed design is scaled to the export size, so a 4K PNG is the cloud you previewed, not a different one.

10. Research honesty — frequency is not a theme

> **A word cloud shows what was said most often. It does not show what mattered most.** Frequency analysis counts surface words; qualitative thematic analysis — coding responses, resolving synonyms and disagreements, weighing a rare but crucial answer — is human research work that no counting tool performs.

This distinction is built into the product, not just this guide. The app itself carries the note that frequency is not validated qualitative themes, because the failure mode is real: a researcher who uploads 200 survey responses, exports the biggest words, and reports them as "the themes" has skipped the analysis. Used honestly, the cloud is an excellent first instrument — it orients the researcher, surfaces vocabulary, and suggests codes to test. The boost and highlight controls exist for presentation; the frequency table always keeps the true counts visible next to them, so a styled cloud can never quietly rewrite the data.

11. The AI roadmap and deployment plan

The brief for this product says "AI-powered", and the marketing temptation is to let a frequency counter wear that label. It would not be true, so version 1.0 does not claim it — and this chapter is the staged, honest route to the features that would earn it.

11.1 What genuinely needs a model

- **AI keyword extraction** — identifying meaningful concepts (including ones never stated verbatim) requires a language model or NLP service. Counting cannot do it.
- **Sentiment scoring** — judging that "the canteen was spotless" is positive and "the food was cold again" is negative, in context, is model work.

11.2 Route A — optional serverless backend proxy (recommended)

A single small serverless function (e.g. on the hosting platform) holds the API key for an NLP/LLM provider in its own environment variables — the key never appears in the HTML, where anyone could read it. The flow is consent-first: the page gains an "AI analysis" switch, off by default; only when the user turns it on and confirms are the words (or a truncated sample of the text) sent over HTTPS to the proxy, which returns extracted keywords and sentiment scores that the cloud then renders. Indicative costs are per-request fractions of a cent for keyword extraction on a small model; the function is stateless and stores nothing. Deployment steps: (1) create the provider account and key; (2) deploy the function with the key as an environment secret; (3) add the opt-in UI and consent copy to the app; (4) label AI-derived output as AI-derived wherever it appears.

11.3 Route B — on-device model (standalone version)

A small transformer or keyword model loaded from a CDN library in the standalone hosted version keeps text entirely on the device. The trade-offs are a heavier first download (tens of MB), slower inference on low-end phones, and a smaller model than any hosted API. It suits the standalone site, not the sandboxed catalogue iframe.

11.4 Sentiment colours — the honest intermediate

Before either route ships, sentiment can be offered as a **clearly-labelled lexicon tint**: built-in positive/negative word lists colour matching words, with the interface naming it exactly that — "simple word-list tint", never "AI sentiment". It is useful, transparent about its limits (sarcasm and negation defeat it), and removable in one click.

11.5 Save projects — why it is standalone-only

Saving editable designs requires browser storage (IndexedDB). The Asif SaaS Studio catalogue runs apps inside a sandboxed iframe under a strict no-storage rule — the same rule that keeps every studio app private and portable — so projects are staged for the standalone hosted version only, where IndexedDB is permitted. The iframe version will keep exporting PNG/SVG/PDF/CSV as its "save".

11.6 Version 1.1 — pure front-end, no backend needed

- **Comparison mode** — two documents side by side, shared words aligned, differences highlighted.
- **Batch generation** — one cloud per row-group of an uploaded CSV/XLSX dataset, downloaded as a set.
- **Weighted word lists** — first-class upload of word,count pairs (already works via CSV tokenising; v1.1 makes it explicit).

12. Publishing and SEO notes

On the current Asif SaaS Studio setup, Word Cloud Studio lives in the tools catalogue (Wix CMS SaasProducts) like every other studio app, under the listing in Chapter 14. The recommended standalone page — `/word-cloud-generator/` with the product title "Free Word Cloud Generator Online – Create & Download Custom Word Clouds", an FAQ section and structured data — belongs to the future public SaaS Studio site build, where real URLs, a sitemap and per-tool pages exist. It is recorded here so the plan is not lost; it is not claimed as done. Separate pages (Text to Word Cloud, Document to Word Cloud, Word Frequency Analyzer) should only be created where they offer distinct functionality, not as duplicated content.

Upload steps (Wix CMS)

1. Wix dashboard → CMS → **SaasProducts** → Add Item.

2. Fill the listing fields from Chapter 14; paste the full contents of `index.html` into **appHtml**.
3. Set **published** on, save, then open the live app page and run the checklist in the test report.

13. How correctness was verified

45 automated checks, all passed, run against the shipped code (not a reimplementation):

- **Frequency engine** — on the prototype text: "research" ×4, "education"/"innovation" ×2, 9 unique words with phrases off and 10 with phrases on (the bigram "science learning" ×2 correctly detected and flagged).
- **Filters** — stop-word removal on/off, number removal on/off, minimum-length filter, phrase detection ("food safety" ×3 in a test paragraph).
- **Multilingual** — Urdu (3× کہانا) and Arabic (3× العلم) tokens counted correctly.
- **Shape masks** — point tests for circle, heart, star, cloud, bulb and tree: centres inside, corners outside, the tree trunk inside.
- **Layout** — with a reference text measurer, all 9 test words placed in rectangle mode with **zero overlapping boxes**; largest word placed first; circle mode places at least 7 of 9 with every centre inside the shape.
- **Real file fixtures** — DOCX (python-docx), XLSX (openpyxl) and PDF (ReportLab) files generated by real tools were extracted to the expected text by the app's own in-page parsers.
- **Export engines** — the zlib writer round-trips through a standard decompressor; the image-PDF writer produces a valid one-page PDF (612 × 459 pts for a 4×3 test image); the SVG contains the words and background; the frequency CSV has the correct header and values.

Two test expectations were corrected during testing — phrase entries raise the unique-word count, and a tight circle honestly reports unplaced words. In both cases the app was right and the test was wrong; that is recorded in the test report rather than smoothed over.

14. Product information and credits

Field	Detail
Product	Word Cloud Studio — Free Word Cloud Generator
Version	1.0 (20 October 2026)
Creator	**Dr. Asif Ahmad**
Publisher	Asif SaaS Studio
Category / slug	Design / word-cloud-studio
Summary	Turn text and documents into beautiful word clouds — frequency analysis, phrases, custom shapes & silhouettes, live design controls, 4K PNG/JPG/SVG/ PDF export. Private: everything runs in your browser.
Deliverable	index.html — 47.9 KB, single file, all CSS and JavaScript inline
Requirements	Any modern browser; no server, no installation, no account
Verification	45/45 automated checks against the shipped cores, including real DOCX/XLSX/ PDF fixtures (Chapter 13)

Appendix A — the analysis & layout core, verbatim

The code below is the complete PURE CORE of the product exactly as it ships inside `index.html`: the stop-word lists, the tokeniser and frequency analyser with phrase detection, the palettes, the shape masks, the spiral layout engine, and the SVG builder. Interface wiring is omitted.

```
/* ---------------- Stop words (basic lists) ---------------- */
var STOP_EN =
("a,about,above,after,again,against,all,am,an,and,any,are,aren,as,at,be,because,been,before,being,below,between,
both,but,by,could,did,do,does,doing,down,during,each,few,for,from,further,had,has,have,having,he,her,here,hers,h
erself,him,himself,his,how,i,if,in,into,is,it,its,itself,let,me,more,most,my,of,off,on,once,only,or,other,our,ou
rs,ourselves,out,over,own,s,same,shan,she,should,so,some,such,t,than,that,the,their,theirs,them,themselves,then,
there,these,they,this,those,through,to,too,under,until,up,very,was,we,were,what,when,where,which,while,who,whom,
why,will,with,would,you,your,yours,yourself,yourselves,also,one,two,get,like,just,can,could,may,might,must,shall
,per,via,etc").split(",");
var STOP_UR =
("اور,میں,ہے,کے,سے,کو,یہ,وہ,بھی,پر,کا,کی,نے,ایک,جو,تک,نہیں,ہیں,تھا,تھی,تھے,ہم,تم,آپ,اس,ان,جب,تو,پھر,اب,کیا,کیوں,کیسے
").split(",");
var STOP_AR =
("في,من,إلى,على,أن,إن,ما,لا,لم,هو,هي,كان,مع,هذا,هذه,ذلك,التي,الذي,عن,بعد,عند,لدى,بين,غير,أي,كل,لأن,لكن,ثم,أو,و,يا
,إذا,قد,لقد,ذو,ذات,نحو,لدى").split(",");
var STOP_SET = {};
STOP_EN.concat(STOP_UR, STOP_AR).forEach(function(w){ STOP_SET[w] = 1; });

/* ---------------- Tokenizing & frequency ---------------- */
/* Word = run of Unicode letters/marks with optional inner apostrophe; numbers are separate tokens. */
var TOKEN_RE = /[\p{L}\p{M}]+(?:['\u2019][\p{L}\p{M}]+)?|\d+(?:[.,]\d+)?/gu;
function tokenize(text){
  if (!text) return [];
  var m = String(text).toLowerCase().match(TOKEN_RE);
  return m || [];
}
```

```
function isNumberToken(t){ return /^\d/.test(t); }
function analyze(text, opts){
  var tokens = tokenize(text);
  var total = tokens.length;
  var freq = {}, kept = [];
  tokens.forEach(function(t){
    if (isNumberToken(t)){
      if (opts.removeNumbers) return;
    } else {
      if (t.replace(/['\u2019]/g, "").length < opts.minLen) return;
      if (opts.removeStop && STOP_SET[t]) return;
    }
    freq[t] = (freq[t] || 0) + 1;
    kept.push(t);
  });
  var phrases = {};
  if (opts.phrases){
    for (var i = 0; i + 1 < kept.length; i++){
      var a = kept[i], b = kept[i + 1];
      if (isNumberToken(a) || isNumberToken(b)) continue;
      var key = a + " " + b;
      phrases[key] = (phrases[key] || 0) + 1;
    }
    Object.keys(phrases).forEach(function(k){
      if (phrases[k] >= 2) freq[k] = phrases[k]; else delete phrases[k];
    });
  }
  var entries = Object.keys(freq).map(function(w){ return { word:w, count:freq[w], phrase: w.indexOf(" ") >
0 }; });
  entries.sort(function(x, y){ return y.count - x.count || (x.word < y.word ? -1 : 1); });
  return { total: total, unique: entries.length, entries: entries, tokensKept: kept.length };
}

/* ---------------- Palettes ---------------- */
var PALETTES = {
  ocean:   ["#0B5394", "#0077B6", "#2E86AB", "#48CAE4", "#00B4D8"],
  studio:  ["#103E32", "#1B7A5A", "#57A773", "#C9A227", "#7FB069"],
  sunset:  ["#9D0208", "#DC2F02", "#E85D04", "#F48C06", "#FAA307"],
  berry:   ["#5F0F40", "#9A031E", "#E36414", "#FB8B24", "#0F4C5C"],
  royal:   ["#7A5C00", "#C9A227", "#8C6D1F", "#B8860B", "#5C4A00"],
  mono:    ["#141414", "#2E2E2E", "#4A4A4A", "#6E6E6E", "#8F8F8F"],
  festival:["#EF476F", "#F78C6B", "#FFD166", "#06D6A0", "#118AB2"]
};
function paletteColor(palette, i, rand){
  var base = palette[i % palette.length];
  return base;
}

/* ---------------- Shape masks (point tests in a W x H box) ---------------- */
function inPolygon(x, y, pts){
  var inside = false;
  for (var i = 0, j = pts.length - 1; i < pts.length; j = i++){
    var xi = pts[i][0], yi = pts[i][1], xj = pts[j][0], yj = pts[j][1];
    if (((yi > y) !== (yj > y)) && (x < (xj - xi) * (y - yi) / (yj - yi) + xi)) inside = !inside;
  }
  return inside;
}
function starPoints(cx, cy, rO, rI){
  var pts = [];
  for (var i = 0; i < 10; i++){
    var r = i % 2 === 0 ? rO : rI, a = -Math.PI / 2 + i * Math.PI / 5;
    pts.push([cx + r * Math.cos(a), cy + r * Math.sin(a)]);
  }
  return pts;
}
var MASKS = {
  rect: function(x, y, W, H){ return x >= 2 && y >= 2 && x <= W - 2 && y <= H - 2; },
  circle: function(x, y, W, H){
    var dx = (x - W / 2) / (W * 0.49), dy = (y - H / 2) / (H * 0.49);
    return dx * dx + dy * dy <= 1;
  },
  heart: function(x, y, W, H){
    var nx = (x - W / 2) / (W * 0.36), ny = (H * 0.58 - y) / (H * 0.42);
    var a = nx, b = ny;
    return Math.pow(a * a + b * b - 1, 3) - a * a * b * b * b <= 0 &&
           x >= W * 0.06 && x <= W * 0.94 && y >= H * 0.04 && y <= H * 0.97;
  },
```

```
  star: function(x, y, W, H){
    return inPolygon(x, y, starPoints(W / 2, H / 2, Math.min(W, H) * 0.52, Math.min(W, H) * 0.22));
  },
  cloud: function(x, y, W, H){
    var blobs = [[0.32, 0.58, 0.20], [0.50, 0.42, 0.26], [0.68, 0.58, 0.20], [0.50, 0.66, 0.28]];
    for (var i = 0; i < blobs.length; i++){
      var dx = (x - blobs[i][0] * W) / (blobs[i][2] * W), dy = (y - blobs[i][1] * H) / (blobs[i][2] * W);
      if (dx * dx + dy * dy <= 1) return true;
    }
    return x >= W * 0.16 && x <= W * 0.84 && y >= H * 0.52 && y <= H * 0.78;
  },
  bulb: function(x, y, W, H){
    var dx = (x - W / 2) / (W * 0.30), dy = (y - H * 0.40) / (H * 0.36);
    if (dx * dx + dy * dy <= 1) return true;
    var pts = [[W * 0.40, H * 0.66], [W * 0.60, H * 0.66], [W * 0.565, H * 0.88], [W * 0.435, H * 0.88]];
    return inPolygon(x, y, pts);
  },
  tree: function(x, y, W, H){
    var tiers = [
      [[W * 0.5, H * 0.05], [W * 0.70, H * 0.42], [W * 0.30, H * 0.42]],
      [[W * 0.5, H * 0.28], [W * 0.76, H * 0.66], [W * 0.24, H * 0.66]],
      [[W * 0.5, H * 0.50], [W * 0.82, H * 0.88], [W * 0.18, H * 0.88]]
    ];
    for (var i = 0; i < tiers.length; i++) if (inPolygon(x, y, tiers[i])) return true;
    return x >= W * 0.465 && x <= W * 0.535 && y >= H * 0.86 && y <= H * 0.97;
  }
};
function pointInMask(mask, x, y, W, H, customGrid){
  if (mask === "custom"){
    if (!customGrid) return true;
    var gx = Math.floor(x / W * customGrid.w), gy = Math.floor(y / H * customGrid.h);
    if (gx < 0 || gy < 0 || gx >= customGrid.w || gy >= customGrid.h) return false;
    return customGrid.cells[gy * customGrid.w + gx] === 1;
  }
  var fn = MASKS[mask] || MASKS.rect;
  return fn(x, y, W, H);
}
function boxInMask(mask, cx, cy, w, h, W, H, customGrid){
  var pts = [[cx, cy], [cx - w / 2, cy - h / 2], [cx + w / 2, cy - h / 2], [cx - w / 2, cy + h / 2], [cx + w /
2, cy + h / 2], [cx, cy - h / 2], [cx, cy + h / 2], [cx - w / 2, cy], [cx + w / 2, cy]];
  for (var i = 0; i < pts.length; i++) if (!pointInMask(mask, pts[i][0], pts[i][1], W, H, customGrid)) return
false;
  return true;
}

/* ---------------- Layout engine ----------------
   entries: [{word, count}] sorted desc. measure(word, sizePx, fontSpec) -> width px.
   Returns placed words with x, y (centre), size, rotate (0|-90), color. */
function lcg(seed){ var s = seed >>> 0 || 1; return function(){ s = (s * 1664525 + 1013904223) >>> 0; return s /
4294967296; }; }
function layoutWords(entries, opts, measure){
  var W = opts.width, H = opts.height;
  var rand = lcg(opts.seed || 42);
  var placed = [], boxes = [];
  if (!entries.length) return placed;
  var maxC = entries[0].count, minC = entries[entries.length - 1].count;
  var maxSize = Math.min(W, H) * 0.19, minSize = Math.max(11, Math.min(W, H) * 0.026);
  function sizeFor(count){
    if (maxC === minC) return (maxSize + minSize) / 2;
    var t = Math.pow((count - minC) / (maxC - minC), 0.55);
    return minSize + t * (maxSize - minSize);
  }
  entries.forEach(function(e, idx){
    var size = sizeFor(e.count) * (e.boost || 1);
    var rotate = 0;
    if (opts.orientation === "v") rotate = -90;
    else if (opts.orientation === "mix") rotate = rand() < 0.28 ? -90 : 0;
    var w = measure(e.word, size, opts), h = size * (opts.tallScript ? 1.75 : 1.06);
    var bw = rotate === 0 ? w : h, bh = rotate === 0 ? h : w;
    var found = null;
    var cx = W / 2, cy = H / 2;
    var maxR = Math.max(W, H);
    for (var r = 0; r < maxR && !found; r += 3){
      var steps = r === 0 ? 1 : Math.max(8, Math.floor(2 * Math.PI * r / 9));
      for (var sIdx = 0; sIdx < steps; sIdx++){
        var a = (sIdx / steps) * Math.PI * 2 + idx * 0.7;
        var x = cx + r * Math.cos(a) * (W / H > 1 ? 1.25 : 1), y = cy + r * Math.sin(a) * 0.82;
```

```
        if (x - bw / 2 < 1 || x + bw / 2 > W - 1 || y - bh / 2 < 1 || y + bh / 2 > H - 1) continue;
        if (!boxInMask(opts.shape, x, y, bw, bh, W, H, opts.customGrid)) continue;
        var hit = false;
        for (var b = 0; b < boxes.length; b++){
          var B = boxes[b];
          if (Math.abs(x - B.x) * 2 < bw + B.w + opts.padding * 2 && Math.abs(y - B.y) * 2 < bh + B.h +
opts.padding * 2){ hit = true; break; }
        }
        if (!hit){ found = { x:x, y:y }; break; }
      }
    }
    if (found){
      boxes.push({ x:found.x, y:found.y, w:bw, h:bh });
      placed.push({ word:e.word, count:e.count, x:found.x, y:found.y, size:size, rotate:rotate,
        color: e.highlight ? "#C9A227" : paletteColor(opts.palette, idx, rand), w:bw, h:bh });
    }
  });
  return placed;
}

/* ---------------- SVG export ---------------- */
function xmlEsc(s){ return String(s).replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g,
"&gt;").replace(/"/g, "&quot;"); }
function buildSVG(placed, opts){
  var W = opts.width, H = opts.height, parts = [];
  parts.push('<svg xmlns="http://www.w3.org/2000/svg" width="' + W + '" height="' + H + '" viewBox="0 0 ' + W +
' ' + H + '">');
  if (opts.background && opts.background !== "transparent") parts.push('<rect width="' + W + '" height="' + H +
'" fill="' + opts.background + '"/>');
  placed.forEach(function(p){
    var rot = p.rotate ? ' transform="rotate(' + p.rotate + ' ' + p.x.toFixed(1) + ' ' + p.y.toFixed(1) + ')"' :
"";
    parts.push('<text x="' + p.x.toFixed(1) + '" y="' + p.y.toFixed(1) + '" font-family="' +
xmlEsc(opts.fontFamily) + '" font-size="' + p.size.toFixed(1) + '" font-weight="' + opts.fontWeight + '" fill="'
+ p.color + '" text-anchor="middle" dominant-baseline="central"' + rot + '>' + xmlEsc(p.word) + '</text>');
  });
  parts.push("</svg>");
  return parts.join("\n");
}
function freqToCSV(entries, total){
  var lines = ["word,count,percent_of_kept_words"];
  entries.forEach(function(e){
    var pctTxt = total > 0 ? (e.count / total * 100).toFixed(2) : "0";
    var w = /[",\n]/.test(e.word) ? '"' + e.word.replace(/"/g, '""') + '"' : e.word;
    lines.push(w + "," + e.count + "," + pctTxt);
  });
  return lines.join("\r\n");
}
```

Appendix B — the file-parsing and export cores, verbatim

First, the IO CORE: the ZIP reader, the DOCX and XLSX extractors, and the PDF text extractor (Flate and ASCII85+Flate). Then the EXPORT CORE: the zlib writer and the image-PDF writer used by the high-resolution exports.

```
/* ---------------- ZIP reading (stored + deflate, via DecompressionStream) ---------------- */
async function inflateBytes(bytes, format){
  var ds = new DecompressionStream(format);
  var stream = new Blob([bytes]).stream().pipeThrough(ds);
  var buf = await new Response(stream).arrayBuffer();
  return new Uint8Array(buf);
}
async function unzipEntries(buf){
  var u8 = new Uint8Array(buf), dv = new DataView(buf);
  var eocd = -1;
  for (var i = u8.length - 22; i >= 0 && i > u8.length - 22 - 66000; i--){
    if (dv.getUint32(i, true) === 0x06054b50){ eocd = i; break; }
  }
  if (eocd < 0) throw new Error("Not a valid ZIP-based file.");
```

```
  var count = dv.getUint16(eocd + 10, true), off = dv.getUint32(eocd + 16, true);
  var map = {};
  for (var n = 0; n < count; n++){
    if (dv.getUint32(off, true) !== 0x02014b50) break;
    var method = dv.getUint16(off + 10, true), csize = dv.getUint32(off + 20, true);
    var nlen = dv.getUint16(off + 28, true), elen = dv.getUint16(off + 30, true), clen = dv.getUint16(off + 32,
true);
    var name = new TextDecoder().decode(u8.slice(off + 46, off + 46 + nlen));
    var lho = dv.getUint32(off + 42, true);
    var lnlen = dv.getUint16(lho + 26, true), lelen = dv.getUint16(lho + 28, true);
    var dataStart = lho + 30 + lnlen + lelen;
    var raw = u8.slice(dataStart, dataStart + csize);
    map[name] = { method: method, raw: raw };
    off += 46 + nlen + elen + clen;
  }
  var out = {};
  for (var key in map){
    var e = map[key];
    if (e.method === 0) out[key] = e.raw;
    else if (e.method === 8) out[key] = await inflateBytes(e.raw, "deflate-raw");
    else throw new Error("Unsupported ZIP compression in file.");
  }
  return out;
}
function decodeEntities(s){
  return s.replace(/&lt;/g, "<").replace(/&gt;/g, ">").replace(/&quot;/g, '"').replace(/&apos;/g, "'").replace(/
&#(\d+);/g, function(_, d){ return String.fromCharCode(Number(d)); }).replace(/&amp;/g, "&");
}
async function extractDocxText(buf){
  var files = await unzipEntries(buf);
  var xml = files["word/document.xml"];
  if (!xml) throw new Error("No document body found in this DOCX file.");
  var s = new TextDecoder("utf-8").decode(xml);
  s = s.replace(/<\/w:p>/g, "\n").replace(/<w:tab[^>]*\/>/g, " ").replace(/<[^>]+>/g, "");
  return decodeEntities(s);
}
async function extractXlsxText(buf){
  var files = await unzipEntries(buf);
  var shared = [];
  if (files["xl/sharedStrings.xml"]){
    var ss = new TextDecoder("utf-8").decode(files["xl/sharedStrings.xml"]);
    var siParts = ss.split(/<si>/).slice(1);
    siParts.forEach(function(part){
      var texts = [], re = /<t[^>]*>([\s\S]*?)<\/t>/g, m;
      while ((m = re.exec(part.split("</si>")[0])) !== null) texts.push(m[1]);
      shared.push(decodeEntities(texts.join("")));
    });
  }
  var sheetName = Object.keys(files).filter(function(k){ return /^xl\/worksheets\/sheet\d+
\.xml$/.test(k); }).sort()[0];
  if (!sheetName) throw new Error("No worksheet found in this XLSX file.");
  var sx = new TextDecoder("utf-8").decode(files[sheetName]);
  var rows = [], rowRe = /<row[^>]*>([\s\S]*?)<\/row>/g, rm;
  while ((rm = rowRe.exec(sx)) !== null){
    var cells = [], cRe = /<c[^>]*?(?:\st="(\w+)")?[^>]*>(?:<v>([\s\S]*?)<\/v>|<is><t[^>]*>([\s\S]*?)<\/t><\/
is>)?<\/c>/g, cm;
    while ((cm = cRe.exec(rm[1])) !== null){
      var val = "";
      if (cm[3] !== undefined) val = decodeEntities(cm[3]);
      else if (cm[2] !== undefined) val = (cm[1] === "s") ? (shared[Number(cm[2])] || "") : cm[2];
      if (val !== "") cells.push(val);
    }
    if (cells.length) rows.push(cells.join(" "));
  }
  return rows.join("\n");
}
/* ---------------- PDF text extraction (Flate streams, Tj/TJ) ---------------- */
function ascii85Decode(str){
  var out = [], group = [];
  var clean = str.replace(/\s/g, "").replace(/^<~/, "").replace(/~>$/, "");
  function flush(g, n){
    var val = 0;
    for (var i = 0; i < 5; i++) val = val * 85 + ((g[i] || 117) - 33);
    var bytes = [(val >>> 24) & 255, (val >>> 16) & 255, (val >>> 8) & 255, val & 255];
    for (var j = 0; j < n - 1; j++) out.push(bytes[j]);
  }
  for (var i = 0; i < clean.length; i++){
```

```
    var ch = clean[i];
    if (ch === "z"){ out.push(0, 0, 0, 0); continue; }
    group.push(ch.charCodeAt(0));
    if (group.length === 5){ flush(group, 5); group = []; }
  }
  if (group.length) { var n = group.length; while (group.length < 5) group.push(117); flush(group, n); }
  return new Uint8Array(out);
}
function pdfContentText(content){
  var out = [], i = 0, L = content.length;
  function readLiteral(start){
    var s = "", j = start + 1, depth = 1;
    while (j < L && depth > 0){
      var c = content[j];
      if (c === "\\"){
        var nx = content[j + 1];
        if (nx === "n") s += "\n"; else if (nx === "r") s += ""; else if (nx === "t") s += " ";
        else if (nx === "(" || nx === ")" || nx === "\\") s += nx;
        else if (nx >= "0" && nx <= "7"){ var oct = nx, k = j + 2; while (k < L && content[k] >= "0" &&
content[k] <= "7" && oct.length < 3){ oct += content[k]; k++; } s += String.fromCharCode(parseInt(oct, 8)); j =
k - 2; }
        else s += nx;
        j += 2; continue;
      }
      if (c === "(") depth++;
      if (c === ")"){ depth--; if (depth === 0){ j++; break; } }
      s += c; j++;
    }
    return { text: s, end: j };
  }
  while (i < L){
    var ch = content[i];
    if (ch === "("){
      var r = readLiteral(i);
      var after = content.slice(r.end, r.end + 12);
      if (/^\s*(Tj|TJ|'|")/.test(after) || /\]\s*TJ/.test(content.slice(Math.max(0, i - 400), i) + "§"))
out.push(r.text);
      else if (/TJ/.test(content.slice(r.end, r.end + 400))) out.push(r.text);
      i = r.end; continue;
    }
    i++;
  }
  return out.join(" ");
}
async function extractPdfText(buf){
  var u8 = new Uint8Array(buf);
  var latin = "";
  for (var i = 0; i < u8.length; i += 8192) latin += String.fromCharCode.apply(null, u8.subarray(i, i + 8192));
  var texts = [], re = /stream\r?\n/g, m;
  while ((m = re.exec(latin)) !== null){
    var start = m.index + m[0].length;
    var end = latin.indexOf("endstream", start);
    if (end < 0) break;
    var rawEnd = end;
    if (latin[rawEnd - 1] === "\n") rawEnd--;
    if (latin[rawEnd - 1] === "\r") rawEnd--;
    var bytes = u8.slice(start, rawEnd);
    var content = null;
    try { content = new TextDecoder("latin1").decode(await inflateBytes(bytes, "deflate")); }
    catch (e1) {
      try { content = new TextDecoder("latin1").decode(await inflateBytes(ascii85Decode(new
TextDecoder("latin1").decode(bytes)), "deflate")); }
      catch (e2) { content = null; }
    }
    if (content && (content.indexOf("Tj") > -1 || content.indexOf("TJ") > -1))
texts.push(pdfContentText(content));
    re.lastIndex = end + 9;
  }
  var joined = texts.join("\n").replace(/\s+/g, " ").trim();
  if (joined.length < 20) throw new Error("This PDF's text could not be extracted — it may be scanned or use
special fonts. Please copy and paste the text instead.");
  return joined;
}
```

```
/* ---------------- zlib (stored blocks) + image PDF ---------------- */
function adler32(bytes){
  var a = 1, b = 0;
  for (var i = 0; i < bytes.length; i++){ a = (a + bytes[i]) % 65521; b = (b + a) % 65521; }
  return ((b << 16) | a) >>> 0;
}
function zlibStore(bytes){
  var out = [0x78, 0x01];
  var pos = 0;
  do {
    var len = Math.min(65535, bytes.length - pos);
    var last = (pos + len >= bytes.length) ? 1 : 0;
    out.push(last, len & 255, (len >>> 8) & 255, (~len) & 255, ((~len) >>> 8) & 255);
    for (var i = 0; i < len; i++) out.push(bytes[pos + i]);
    pos += len;
  } while (pos < bytes.length);
  var ad = adler32(bytes);
  out.push((ad >>> 24) & 255, (ad >>> 16) & 255, (ad >>> 8) & 255, ad & 255);
  return new Uint8Array(out);
}
function buildImagePDF(rgb, w, h){
  var raw = new Uint8Array(h * (1 + w * 3));
  var src = 0, dst = 0;
  for (var y = 0; y < h; y++){
    raw[dst++] = 0;
    for (var x = 0; x < w * 3; x++) raw[dst++] = rgb[src++];
  }
  var z = zlibStore(raw);
  var pageW = 612, pageH = Math.round(612 * h / w);
  var parts = [], offsets = [], cursor = 0;
  function pushStr(s){ parts.push(s); cursor += s.length; }
  function pushBytes(b){ parts.push(b); cursor += b.length; }
  pushStr("%PDF-1.4\n");
  offsets[1] = cursor; pushStr("1 0 obj\n<< /Type /Catalog /Pages 2 0 R >>\nendobj\n");
  offsets[2] = cursor; pushStr("2 0 obj\n<< /Type /Pages /Kids [3 0 R] /Count 1 >>\nendobj\n");
  offsets[3] = cursor; pushStr("3 0 obj\n<< /Type /Page /Parent 2 0 R /MediaBox [0 0 " + pageW + " " + pageH +
"] /Resources << /XObject << /Im1 4 0 R >> >> /Contents 5 0 R >>\nendobj\n");
  offsets[4] = cursor; pushStr("4 0 obj\n<< /Type /XObject /Subtype /Image /Width " + w + " /Height " + h + " /
ColorSpace /DeviceRGB /BitsPerComponent 8 /Filter /FlateDecode /Length " + z.length + " >>\nstream\n");
  pushBytes(z); pushStr("\nendstream\nendobj\n");
  var content = "q " + pageW + " 0 0 " + pageH + " 0 0 cm /Im1 Do Q";
  offsets[5] = cursor; pushStr("5 0 obj\n<< /Length " + content.length + " >>\nstream\n" + content +
"\nendstream\nendobj\n");
  var xrefPos = cursor;
  var tail = "xref\n0 6\n0000000000 65535 f \n";
  for (var n = 1; n <= 5; n++){ var off = String(offsets[n]); while (off.length < 10) off = "0" + off; tail +=
off + " 00000 n \n"; }
  tail += "trailer\n<< /Size 6 /Root 1 0 R >>\nstartxref\n" + xrefPos + "\n%%EOF";
  pushStr(tail);
  var total = 0; parts.forEach(function(p){ total += (typeof p === "string") ? p.length : p.length; });
  var outArr = new Uint8Array(total), o = 0;
  parts.forEach(function(p){
    if (typeof p === "string"){ for (var i = 0; i < p.length; i++) outArr[o++] = p.charCodeAt(i) & 255; }
    else { outArr.set(p, o); o += p.length; }
  });
  return outArr;
}
```